

LIVELLI DI VISIBILITÀ DI UN ATTRIBUTO

I livelli di visibilità di un attributo stabiliscono se l'attributo è accessibile ad altre classi, cioè se all'esterno della classe che lo contiene si può leggere o modificare il suo valore.

Tre livelli di visibilità:

- **public**: attributo accessibile da qualsiasi altra classe;
- **private**: attributo visibile solo dalla classe che lo contiene: unicamente la classe che contiene l'attributo può modificarne il valore;
- **protected**: attributo accessibile dalla classe che lo contiene e dalle sue sottoclassi, ovvero le classi che ereditano da essa.

NOTA BENE: public, private e protected sono **MODIFICATORI DI ACCESSO**.

Spesso il modificatore di accesso protected viene utilizzato per definire gli attributi delle classi ASTRATTE, ovvero quelle classi che non possono essere implementate, ma devono necessariamente essere derivate.

NOTA BENE: **UNA CLASSE ASTRATTA NON PUÒ ESSERE ISTANZIATA DIRETTAMENTE.**

Cos'è un modificatore?

Un modificatore è una parola-chiave che cambia il comportamento delle variabili, dei metodi o delle classi.

Esempio di classe astratta:

```
public abstract class Figura {  
    .....  
    .....  
}  
  
public class Cerchio extends Figura {  
    .....  
    .....  
}
```

MODIFICATORE DI TIPO RESTRITTIVO

Il modificatore **final** è di tipo restrittivo. Si può applicare agli variabili (attributi), ai metodi e alle classi.

- **Variabili:** quando applicata ad una variabile (attributo), final rende quella variabile una costante.
Es. `final int costante = 10;`
- **Metodi:** quando un metodo è dichiarato final non può essere **sovrascritto dalle sottoclassi** (classe figlie/derivate), ovvero una classe derivata non può fornire la propria versione di quel metodo.

Esempio:

```
public class Base {
    //attributi
    ...
    ...
    //Metodi
    public final void metodo()
    {
        ...
        ...
    }
}

public class Derivata extends Base {
    //attributi
    ...
    ...
    //Metodi
    public void metodo()
    {
        ...
        ...
    }
}
```

ERRORE: NON SI PUÒ EFFETTUARE LA SOVRASCRITTURA PERCHÉ IL METODO ALL'INTERNO DELLA SUPERCLASSE È STATO DEFINITO FINAL.

- **Classi:** quando una classe è dichiarata `final` non può essere derivata, ovvero non possono esserci sottoclassi.

Esempio:

```
public final class Base {
    .....
    .....
}
public class Derivata extends Base {
    .....
    .....
}
```

ERRORE: NON SI PUÒ FARE!

MODIFICATORE DI COMPORTAMENTO: `static`

Gli attributi posso essere definiti statici oppure non statici.

Attributi non statici

Per gli attributi non statici viene creata una copia per ogni oggetto istanziato dalla classe. Ciò significa che ogni istanza ha il proprio set di variabili di istanza che possono assumere valori diversi.

Esempio:

```
public class Persona {
    private String nome;
    private int eta;
    ...
    ...
}
```

```
public class Main {
    public static void main(String[] args){
        Persona persona1=new Persona();
        Persona persona2=new Persona();
    }
}
```

persona1 e **persona2** sono istanze separate della classe Persona, ognuna con la propria copia degli attributi (nome, eta) che possono assumere valori diversi.

NOTA BENE: **ogni oggetto in Java ha la sua copia degli attributi, permettendo loro di avere valori diversi dagli altri oggetti istanziati.**

Attributi statici

Per gli attributi statici esiste una sola copia condivisa fra tutte le istanze della classe. Di conseguenza, qualsiasi modifica apportata a questo attributo da un'istanza è visibile a tutte le altre istanze.

Esempio

```
public static final PI;
```

(attributo statico, costante e accessibile anche all'esterno della classe in cui è stato definito).

Esempio:

```
public class Contatore {
    private static int contatore = 0;
    public Contatore() {
        contatore++;
    }
    public void mostraContatore() {
        System.out.println("Valore del contatore: " + contatore);
    }
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Contatore c1 = new Contatore(); // contatore = 1  
        Contatore c2 = new Contatore(); // contatore = 2  
        Contatore c3 = new Contatore(); // contatore = 3  
  
        c1.mostraContatore(); // Output: Valore del contatore: 3  
        c2.mostraContatore(); // Output: Valore del contatore: 3  
        c3.mostraContatore(); // Output: Valore del contatore: 3  
    }  
}
```

NOTA BENE:

poiché le variabili (attributi) **static** vengono allocate una sola volta nella memoria, l'uso della memoria è più efficiente rispetto all'allocazione di variabili non statiche per ogni istanza della classe.

DICHIARAZIONE DI UN METODO STATICO

I metodi dichiarati nelle classi si possono distinguere in due tipologie:

- METODI DI ISTANZA
- METODI DI CLASSE

I **metodi di istanza** sono utilizzabili dalle istanze (oggetti) della classe. In pratica, dalla classe si crea un oggetto e, utilizzando l'oggetto si invoca l'esecuzione dei metodi di istanza.

I metodi di istanza vengono invocati sugli oggetti, istanza della classe, mediante l'operatore punto (.) [DOT NOTATION].

I metodi di istanza possono fare riferimento agli attributi dell'oggetto, leggendo o modificando il valore degli attributi.

I **metodi di classe (STATIC)** esistono e possono essere utilizzati anche senza aver istanziato un oggetto della classe che li contiene.

Questi metodi non possono fare riferimento agli attributi definiti nella classe, a meno che non siano definiti come statici.

Esempio 1

```
public class Operazione {
    private int a, b; //attributi non statici
    public Operazione(int a, int b){
        this.a=a;
        this.b=b;
    }
    public int somma() METODO DI ISTANZA
    {
        return a + b;
        //oppure return this.a + this.b;
    }
}
```

```
public class Main(){
    ...
    int risultato;
    Operazione o1 = new Operazione(1, 2);
    risultato = o1.somma();
}
```

Esempio 2

```
public class Operazione {
    ...
    ...
    public static int somma(int a, int b) METODO DI CLASSE
    {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args){
        int risultato;
        risultato = Operazione.somma(2, 3);
        System.out.println("Somma = " + risultato);
    }
}
```

NOTA BENE: Per l'invocazione di un metodo statico è sufficiente scrivere il nome della classe seguito dal nome del metodo statico e dei parametri.

NOTA BENE: nella classe Operazione non è stato definito il costruttore in quanto la classe non viene istanziata.

NOTA BENE: nel codice non sono stati definiti attributi in quanto il metodo somma è stato definito come STATICO e può fare riferimento solo ad attributi di tipo statico.

I METODI STATICI APPARTENGONO ALLA CLASSE STESSA E NON RICHIEDONO LA CREAZIONE DI UN'ISTANZA PER ESSERE UTILIZZATI.

Esempio:

```
public class Massimo {
    public static int max(int a, int b) {
        if(a > b)
            return a;
        else
            return b;
    }
}

Public class Main {
    public static void main(String[] args){
        int num1 = 7;
        int num2 = 10;
        int risultato = Massimo.max(num1, num2);
        System.out.println("Il massimo tra " + num1 + " e
        " + num2 + " e': " + risultato);
    }
}
```

Esempio:

```
public class Biblioteca {
    public static int prenotazioniTotali = 0;

    public static void aggiungiPrenotazione() {
        prenotazioniTotali++;
    }

    public static void mostraPrenotazioniTotali() {
        System.out.println("Prenotazioni totali: " +
        prenotazioniTotali);
    }
}
```



```
public class Main {
    public static void main(String[] args) {
        Biblioteca.aggiungiPrenotazione();
        Biblioteca.aggiungiPrenotazione();
        Biblioteca.aggiungiPrenotazione();

        Biblioteca.mostraPrenotazioniTotali();
        // Uscita: Prenotazioni totali: 3
    }
}
```

Java include numerosi metodi predefiniti che sono di tipo statico.

Metodi statici della classe Math:

- `Math.abs(int a)`: restituisce il valore assoluto di un numero.
- `Math.max(int a, int b)`: restituisce il massimo tra due numeri.
- `Math.min(int a, int b)`: restituisce il minimo tra due numeri.
- `Math.sqrt(double a)`: restituisce la radice quadrata di un numero.
- `Math.pow(double a, double b)`: restituisce il risultato di un numero elevato a una potenza.

Metodi statici della classe String:

- `String.valueOf(int i)`: converte un valore int in una stringa.
- `length()`: Restituisce la lunghezza della stringa.
- `charAt(int index)`: restituisce il carattere all'indice specificato.
- `substring(int beginIndex, int endIndex)`: restituisce una sottostringa compresa tra gli indici specificati.
- `equalsIgnoreCase(String anotherString)`: Confronta la stringa con un'altra stringa ignorando le differenze di maiuscole/minuscole.

Esempio:

```
String stringa1 = "Hello, World!";  
String stringa2 = "hello, world!";  
String confronto;  
confronto = stringa1.equalsIgnoreCase(stringa2);
```

Quando si usano i metodi statici?

Può essere necessario dichiarare un metodo come **static** se questo metodo è pensato per essere invocato non in relazione a uno specifico oggetto di una classe.

Classe Math: i metodi della classe Math (ad esempio Math.sqrt) sono statici e invocano Math, non uno specifico oggetto.